

Final Project

Name of the report: `Final_Project_DevSecOps`

Course: DevOps and Security S26

Performed by: *Roman Solovev, Salekh Mamadaliev, Nikita Niakhai* — M25-SNE-01

Date submission: 16.03.2026

Video Demo: [Link](#)

1. Introduction

Abstract:

This project presents an AI-powered DevSecOps platform that integrates a large language model into a self-hosted GitLab CI/CD pipeline for automated code security review and interactive bot-assisted analysis. The system is deployed on a single hardened VPS using Docker for GitLab CE and Minikube for Kubernetes orchestration of the AI bot service, with Caddy as reverse proxy and HAProxy for load balancing. The result is a working end-to-end solution where every code commit is automatically scanned for vulnerabilities by the LLM, and developers can also request AI feedback directly through GitLab slash commands.

Keywords: CI/CD, DevSecOps, Kubernetes, Docker, GitLab, LLM, AI, Reverse Proxy, Security, Automation

2. Project Overview

The project focuses on CI/CD convenience through an AI-powered web service deeply integrated into a GitLab-based DevSecOps pipeline. It combines a self-hosted GitLab CE instance, a Kubernetes-orchestrated runner and bot (Uvicorn web-server) infrastructure, and LLM-based code analysis into a single cohesive system running on a hardened VPS. The platform enables automated AI security reviews during CI/CD pipeline execution and interactive AI-assisted code review via slash commands in GitLab commits.

2.1 Goals and Motivation

The main goal is to reduce the manual effort and delay involved in code security reviews by embedding an LLM directly into the CI/CD workflow, so developers receive immediate, automated feedback on vulnerabilities and code quality with every commit. Traditional security review processes often happen late in the development cycle or require dedicated personnel, creating bottlenecks — this project addresses that by making AI-driven analysis a native pipeline stage and an on-demand bot accessible through GitLab comments. The project also demonstrates a fully self-hosted, infrastructure-aware DevSecOps setup with proper server hardening, container orchestration, reverse proxying, and load balancing, delivering practical value as both a security tool and a reference architecture for small-team DevOps environments.

2.2 Technology Stack

List and briefly describe each technology used.

Category	Technology	Purpose
CI/CD	GitLab CE	Pipeline orchestration and source control
Containerization	Docker	Application and service packaging
Orchestration	Minikube / kubectl	Local Kubernetes cluster management
Reverse Proxy	Caddy	HTTPS routing to internal services
Load Balancer	HAProxy	Traffic distribution to the AI bot
AI / LLM	Openrouter API + Gitlab API + Gitlab webhooks	Code review, pipeline analysis, bot commands
Security	Fail2ban, UFW, SSH hardening	Server-level protection

Notes

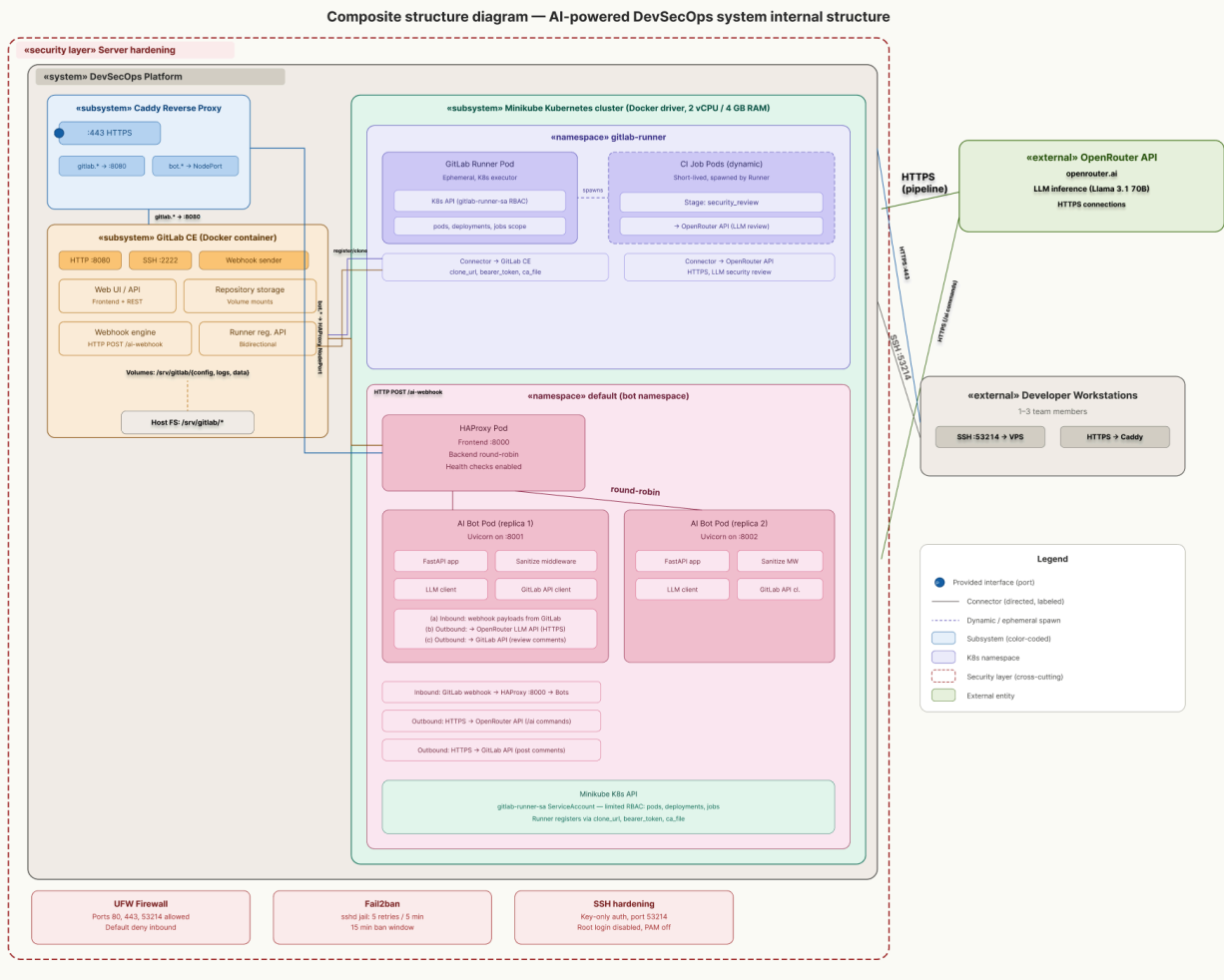


Figure 2.1 — Composite structure UML diagram

Composite Structure Diagram: The diagram shows a top-level "DevSecOps Platform" block containing Caddy (HTTPS :443 facing outward), a GitLab CE Docker container (with Web UI, Repository, and Webhook Engine sub-parts plus volume mount connectors), and a Minikube Cluster subdivided into "gitlab-runner" namespace (Runner + ephemeral CI Job pods) and "default" namespace (HAProxy round-robin to two AI Bot pods), all wrapped in a UFW/Fail2ban/SSH hardening boundary. External actors — Developer Workstations, OpenRouter API, Let's Encrypt, and nip.io — sit outside the system boundary, with connectors going to/from the AI Bot pods (OpenRouter, GitLab API), CI Job pods (OpenRouter), and Caddy (developers).

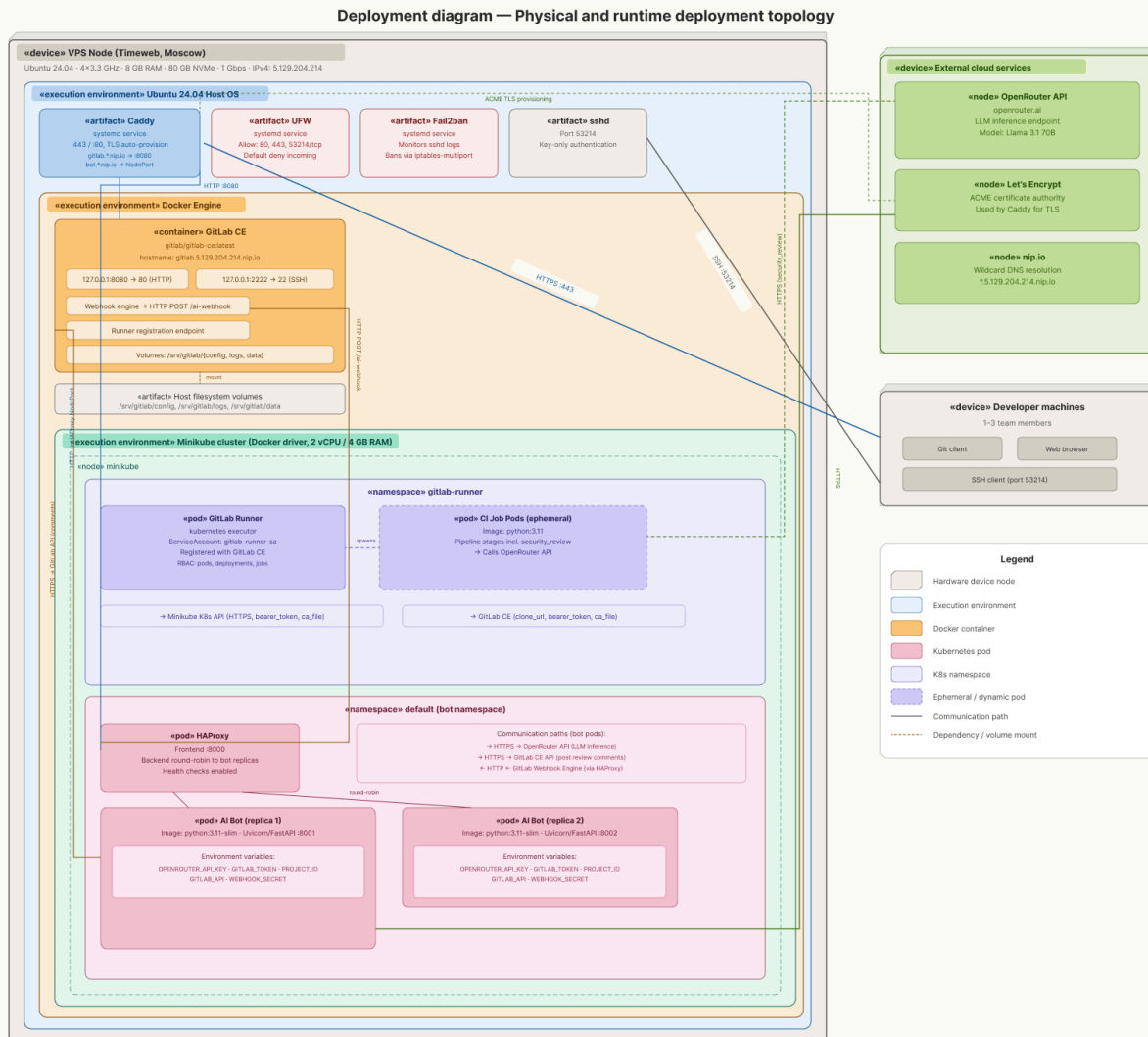


Figure 2.2 — Deployment UML diagram

Deployment Diagram: The diagram shows a single "VPS" hardware node hosting Caddy (:443), UFW, Fail2ban, and sshd (:53214) at the OS level, a Docker Engine containing the GitLab CE container (localhost:8080/2222 with volume dependency arrows to /srv/gitlab/*), and a nested Minikube node (2 vCPU / 4 GB) with GitLab Runner pods in "gitlab-runner" namespace and HAProxy + three AI Bot replica pods in "lb-demo" namespace. Communication paths labeled by protocol/port connect external Developer Machines (SSH :53214, HTTPS :443) into the VPS, Caddy to GitLab and Minikube's HAProxy internally, and Bot/CI pods outward to external cloud nodes (OpenRouter API, Let's Encrypt, nip.io).

3. Architecture Model

The project runs on a **VPS** accessed via SSH with key-based authentication. All services are deployed either directly in Docker or inside a Minikube Kubernetes cluster on the same host.

3.1 Infrastructure Setup

- VPS provider Timeweb
- Price: 1800 RUB / month
- Location: Russia, Moscow
- OS: Ubuntu 24.04
- 4 × 3.3 GH CPU • 8 GB RAM • 80 GB of space NVMe
- Internet configuration: throughput 1000 mb/s, public IPV4 address, nio free reverse domain name for securing https connection

3.2 Keys and access to VPS

`root` user is only available from any other user inside VPS, root login is permitted. One user `project` is granted with all required access privileges and ability to switch on root user.

Execution

```
# Example: SSH key setup
ssh-keygen -t ed25519 -C "devops-final"
ssh-copy-id -i ~/.ssh/id_ed25519.pub user@vps-ip
```

```
Host devops_proj
  HostName 5.129.204.214
  Port 53214
  StrictHostKeyChecking no
  User project
  ForwardAgent yes
  IdentityFile ~/.ssh/devops_proj_nikita
  IdentitiesOnly yes
  AddKeysToAgent yes
```

Figure 3.2 — SSH access and key configuration on developer's machine

```
GNU nano 7.2 /root/.ssh/authorized_keys
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQgQDPVZaeLuL+085y6l89rR1V/j9
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQgQDBVpXdzaqQ0lmbKIfc3BdkWs0
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQgQC4MRq3hjeraRRNtWLEb6FvYA
```

Figure 3.2 — Authorized keys for accessing VPS via `ssh` for developers

4. Server Security

The server is hardened using several complementary tools: **Fail2ban** for brute-force protection, **UFW** as a firewall, and **SSH hardening** to restrict access vectors.

4.1 SSH Hardening

Custom configuration for SSH service were added:

- changed default 22 port to 53214;
- disabled root login;
- disabled password authentication;
- disabled PAM;
- only Public key authentication is available.

```

project@devops-project:~$ cat /etc/ssh/sshd_config.d/10-security.conf
Port 53214
PermitRootLogin no
PasswordAuthentication no
PubkeyAuthentication yes
UsePAM no
project@devops-project:~$ sudo systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: enabled)
   Active: active (running) since Fri 2026-03-13 08:13:08 UTC; 6min ago
 TriggeredBy: ● ssh.socket
     Docs: man:sshd(8)
           man:sshd_config(5)
   Process: 3470736 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
    Main PID: 3470737 (sshd)
       Tasks: 1 (limit: 9483)
      Memory: 1.4M (peak: 11.3M)
         CPU: 108ms
        CGroup: /system.slice/ssh.service
                └─3470737 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Mar 13 08:13:08 devops-project systemd[1]: Starting ssh.service - OpenBSD Secure Shell server...
Mar 13 08:13:08 devops-project sshd[3470737]: Server listening on 0.0.0.0 port 53214.
Mar 13 08:13:08 devops-project sshd[3470737]: Server listening on :: port 53214.
Mar 13 08:13:08 devops-project systemd[1]: Started ssh.service - OpenBSD Secure Shell server.

```

Figure 4.1 — Status of hardened SSH service

Checking that password authentication is indeed disabled:

```

C:\Users\Salik>ssh -p 53214 project@5.129.204.214
project@5.129.204.214: Permission denied (publickey).

```

Figure 4.2 — Checking password authentication

4.2 UFW Firewall Rules

```

project@devops-project:~$ sudo systemctl status ufw
● ufw.service - Uncomplicated firewall
   Loaded: loaded (/usr/lib/systemd/system/ufw.service; enabled; preset: enabled)
   Active: active (exited) since Sat 2026-02-28 12:08:01 UTC; 1 week 5 days ago
     Docs: man:ufw(8)
   Main PID: 504 (code=exited, status=0/SUCCESS)
      CPU: 79ms

Feb 28 12:08:01 devops-project systemd[1]: Starting ufw.service - Uncomplicated firewall...
Feb 28 12:08:01 devops-project systemd[1]: Finished ufw.service - Uncomplicated firewall.
project@devops-project:~$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), deny (routed)
New profiles: skip

To Action From
--
80 ALLOW IN Anywhere
443 ALLOW IN Anywhere
53214/tcp ALLOW IN Anywhere
443/tcp ALLOW IN Anywhere
443/udp ALLOW IN Anywhere
80 (v6) ALLOW IN Anywhere (v6)
443 (v6) ALLOW IN Anywhere (v6)
53214/tcp (v6) ALLOW IN Anywhere (v6)
443/tcp (v6) ALLOW IN Anywhere (v6)
443/udp (v6) ALLOW IN Anywhere (v6)

```

Figure 4.3 — UFW service

We can check open ports on machine using nmapX on other host. Used kali machine for this:

```

# nmap --min-rate=10000 -T4 -sS -p- 5.129.204.214
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-13 04:18 EDT
Nmap scan report for 5.129.204.214
Host is up (0.0080s latency).
Not shown: 65532 filtered tcp ports (no-response)
PORT      STATE SERVICE
80/tcp    open  http
443/tcp    open  https
53214/tcp open  unknown

Nmap done: 1 IP address (1 host up) scanned in 14.45 seconds

```

Figure 4.4 — nmap scan of the cloud machine

4.3 Fail2ban Configuration

Fail2ban is configured via `/etc/fail2ban/jail.local` with the following global parameters in the `[DEFAULT]` section:

- `"bantime = 900"` — an offending IP address will be banned for 900 seconds (15 minutes) upon triggering the threshold;
- `"findtime = 300"` — the time window within which failed attempts are counted is 300 seconds (5 minutes);
- `"maxretry = 5"` — IP address is banned after 5 consecutive failed authentication attempts within the time window;
- `"banaction = iptables-multiport"` — banning is enforced via iptables, blocking the IP across multiple ports simultaneously.

The SSH jail is explicitly enabled and configured with the following parameters:

- `"enabled = true"` — SSH jail is active and monitoring for failed authentication attempts;
- `"port = 53214"` — Fail2ban monitors and applies bans on the custom SSH port 53214;
- `"filter = sshd"` — uses the built-in sshd filter to parse SSH-related log entries;
- `"logpath = %(sshd_log)s"` — log file path from the system default SSH log location;
- `"backend = %(sshd_backend)s"` — log monitoring backend is inherited from system defaults;
- `maxretry = 5` — Overrides (though matches) the global default, banning an IP after 5 failed attempts.

The remaining parameters (`bantime`, `findtime`) are inherited from the `[DEFAULT]` section.

`"[DEFAULT]"` section settings apply globally to all enabled jails. The configuration ensures that any IP exceeding 5 failed login attempts within a 5-minute period is automatically blocked for 15 minutes at the firewall level, mitigating brute-force attacks.

```

GNU nano 7.2 /etc/fail2ban/jail.local
[DEFAULT]

bantime = 900
findtime = 300
maxretry = 5
banaction = iptables-multiport

```

Figure 4.5 — Fail2ban configuration "DEFAULT" section

```

GNU nano 7.2 /etc/fail2ban/jail.local
[sshd]

# To use more aggressive sshd modes set filter parameter "mode" in jail.local:
# normal (default), ddos, extra or aggressive (combines all).
# See "tests/files/logs/sshd" or "filter.d/sshd.conf" for usage example and details.
#mode = normal
enabled = true
port = 53214
filter = sshd
logpath = %(sshd_log)s
backend = %(sshd_backend)s
maxretry = 5

```

Figure 4.5 — Fail2ban configuration "sshd" jail section

We can check status of fail2ban and block ip list using command: `fail2ban-client status`. As we are already hardened SSH service, there's no active brute force attempts from the scanners on the internet.

```

project@devops-project:~$ sudo fail2ban-client status sshd
Status for the jail: sshd
|- Filter
|  |- Currently failed: 0
|  |- Total failed:    1
|  `-- File list:      /var/log/auth.log
`- Actions
   |- Currently banned: 0
   |- Total banned:    0
   `-- Banned IP list:

```

Figure 4.6 — Fail2ban status

5. Containerization and Orchestration

GitLab CE runs inside a Docker container on the VPS. The **GitLab Runner** is deployed inside Minikube (Kubernetes) in a dedicated namespace with a scoped access policy, providing isolation between the runner and other cluster workloads.

5.1 GitLab CE in Docker

Community edition of Gitlab is running as a standalone container on the machine, configured to serve UI via a `gitlab.`-prefixed nip.io hostname. Interface and ssh ports are binded to local ports to be later proxied by Caddy outside. Config and log volumes are mapped to preserve all the important data between restarts.

- **Hostname configuration:** Sets the container hostname to `gitlab.5.129.204.214.nip.io`
- **Port mapping:**
 - Binds host port 8080 (localhost only) to container port 80 for HTTP access
 - Binds host port 2222 (localhost only) to container port 22 for SSH access
- **Volume mounts** (persistent data storage):
 - `/srv/gitlab/config:/etc/gitlab` - GitLab configuration files
 - `/srv/gitlab/logs:/var/log/gitlab` - GitLab log files
 - `/srv/gitlab/data:/var/opt/gitlab` - GitLab application data
- **Image:** Uses the latest GitLab Community Edition image from Docker Hub

```
project@devops-project:~$ docker run -d --hostname gitlab.5.129.204.214.nip.io --name gitlab --restart always -p 127.0.0.1:8080:80 -p 127.0.0.1:2222:22 -v /srv/gitlab/config:/etc/gitlab -v /srv/gitlab/logs:/var/log/gitlab -v /srv/gitlab/data:/var/opt/gitlab gitlab/gitlab-ce:latest
```

Figure 5.1 — Command for running Gitlab Server

```

{
  "Command": "\"/assets/init-contai-\"",
  "CreatedAt": "2026-03-05 16:26:20 +0000 UTC",
  "ID": "d0370c9f415b",
  "Image": "gitlab/gitlab-ce:latest",
  "Labels": "org.opencontainers.image.authors=GitLab Distribution Team <distribution-be@gitlab.com>,org.opencontainers.image.base.name=docker.i
o/ubuntu:24.04,org.opencontainers.image.documentation=https://docs.gitlab.com/ee/install/docker/,org.opencontainers.image.ref.name=ubuntu,org.o
pencontainers.image.source=https://gitlab.com/gitlab-org/omnibus-gitlab,org.opencontainers.image.title=GitLab Omnibus Docker,org.opencontainers
.image.version=24.04",
  "LocalVolumes": "0",
  "Mounts": "/srv/gitlab/co.../srv/gitlab/lo.../srv/gitlab/da...",
  "Names": "gitlab",
  "Networks": "bridge",
  "Platform": {
    "architecture": "amd64",
    "os": "linux"
  },
  "Ports": "443/tcp, 127.0.0.1:2222->22/tcp, 127.0.0.1:8080->80/tcp",
  "RunningFor": "7 days ago",
  "Size": "0B",
  "State": "running",
  "Status": "Up 7 days (healthy)"
}

```

Figure 5.2 — GitLab CE running in Docker (docker ps output)

5.2 Minikube Cluster Setup

No extra configurations or plugins (except kubectl) were added to bare minikube setup, it was started with docker as a virtualization driver, 2vCPU/4GBRAM resource limit.

- **Driver specification (`-driver=docker`)**: Configures Minikube to use Docker as the underlying virtualization driver, running Kubernetes nodes as Docker containers
- **CPU allocation (`-cpus=2`)**: Allocates 2 CPU cores to the Minikube virtual machine/container for cluster operations
- **Memory allocation (`-memory=4g`)**: Assigns 4 gigabytes of RAM to the Minikube instance to ensure adequate resources for running Kubernetes components and applications

```

# Minikube init
minikube start --driver=docker --cpus=2 --memory=4g

```

```
[project@devops-project:~$ minikube kubectl get nodes
NAME          STATUS    ROLES          AGE      VERSION
minikube      Ready     control-plane   7d13h    v1.35.1
[project@devops-project:~$ minikube status
minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured
project@devops-project:~$ █
```

Figure 5.3 — Minikube running + node status

5.3 GitLab Runner in Kubernetes

Gitlab runner is registered and configured as a k8s executor, meaning for each job, a separate short-living pod is launched and runs until the job is finished (Figure 5.5).

To separate the “service” pods - our gitlab runs, and other useful workload, all gitlab-related operations and configurations are moved to a separate namespace `gitlab-runner`. Runner is instructed to assume the service account with limited pods/deployment/jobs permissions to manage job pods mentioned above (Figure 5.4). Other resources and any resource outside of its namespace is inaccessible.

```
[project@devops-project:~/minik$ cat runnersa.yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  name: gitlab-runner-sa
  namespace: gitlab-runner
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: gitlab-runner-role
  namespace: gitlab-runner
rules:
- apiGroups: [""] # Core API group
  resources: ["pods", "pods/exec", "pods/log", "secrets", "services", "configmaps", "events", "serviceaccounts"]
  verbs: ["get", "list", "watch", "create", "update", "patch", "delete"]
- apiGroups: ["apps"]
  resources: ["deployments", "statefulsets"]
  verbs: ["get", "list", "watch", "create", "update", "patch", "delete"]
- apiGroups: ["batch"]
  resources: ["jobs"]
  verbs: ["get", "list", "watch", "create", "update", "patch", "delete"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: gitlab-runner-binding
  namespace: gitlab-runner
subjects:
- kind: ServiceAccount
  name: gitlab-runner-sa
  namespace: gitlab-runner
roleRef:
  kind: Role
  name: gitlab-runner-role
  apiGroup: rbac.authorization.k8s.io
project@devops-project:~/minik$
```

Figure 5.4 — k8s Service Account for gitlab runner

```
[project@devops-project:~/minik$ kubectl get pods -n gitlab-runner
NAME                                READY   STATUS    RESTARTS   AGE
runner-t0-dse4dp-project-1-concurrent-0-sdvomedn  2/2    Terminating    0         13s
runner-t0-dse4dp-project-1-concurrent-0-ylewlwst  2/2    Running         0         4s
project@devops-project:~/minik$
```

Figure 5.5 — Runner pods launched on pipeline job runs

As mentioned above, gitlab runner assumes the `gitlab-runner-sa` Service Account, which is configured in its configuration toml file created and edited on runner registration on the server (Figure 5.7)

Configuration toml (Figure 5.6) also includes other keys and crucial configurations for launching the pods and managing the server - k8s executor integration:

Attribute	Description
<code>url</code>	The full URL of the GitLab instance where the runner is registered
<code>token</code>	The unique authentication token used by the runner to authenticate with the GitLab instance

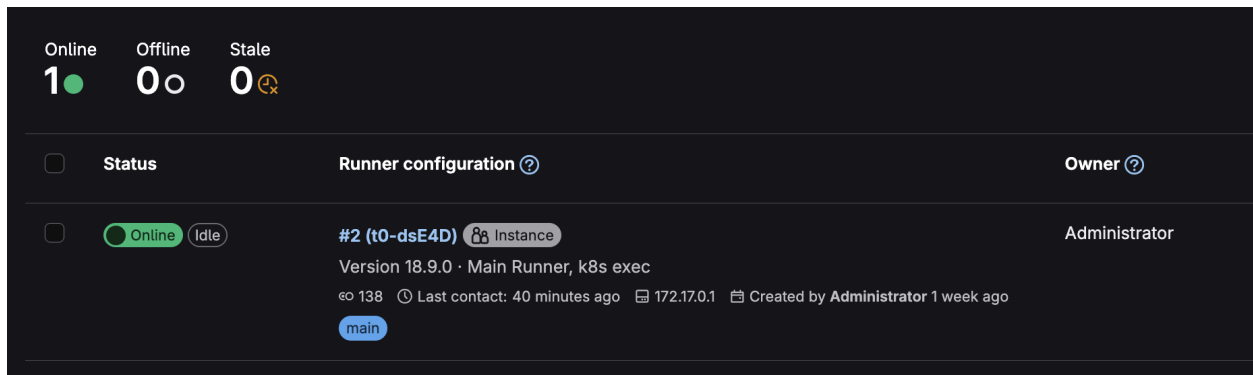


Figure 5.7 — Runner registered and active in GitLab UI

6. Reverse Proxy Configuration

Caddy acts as the entry point for all external HTTPS traffic, routing requests between the internet and internal services — GitLab and the AI bot.

6.1 Caddy Setup

Caddy was installed as a service and provided with a minimal configuration to route traffic to gitlab container and minikube, where load balancing is performed with other more “configured” tools:

```
GNU nano 7.2
gitlab.5.129.204.214.nip.io {
    reverse_proxy 127.0.0.1:8080
}

lb.5.129.204.214.nip.io {
    reverse_proxy 192.168.49.2:30080
}
```

Figure 6.1 — Caddyfile configuration

```

project@devops-project:~$ sudo systemctl status caddy
[sudo] password for project:
Sorry, try again.
[sudo] password for project:
● caddy.service - Caddy
   Loaded: loaded (/usr/lib/systemd/system/caddy.service; enabled; preset: enabled)
   Active: active (running) since Thu 2026-03-05 16:27:44 UTC; 1 week 0 days ago
     Docs: https://caddyserver.com/docs/
   Process: 2322173 ExecReload=/usr/bin/caddy reload --config /etc/caddy/Caddyfile --force (code=exited, status=0/SUCCESS)
   Main PID: 67693 (caddy)
    Tasks: 10 (limit: 9483)
   Memory: 25.9M (peak: 66.8M)
      CPU: 7min 51.850s
   CGroup: /system.slice/caddy.service
           └─67693 /usr/bin/caddy run --environ --config /etc/caddy/Caddyfile

```

Figure 6.2 — Caddy service

6.2 TLS / HTTPS

Caddy also has Automatic HTTPS capability that handles SSL/TLS certificate provisioning and renewal via Let's Encrypt without manual intervention, paired with nip.io - a wildcard DNS service that maps any IP address in a domain name (e.g., `gitlab.5.129.204.214.nip.io` resolves to `5.129.204.214`), eliminating the need for custom domain configuration or local DNS entries results in a simplistic and effective HTTPS+DNS setup

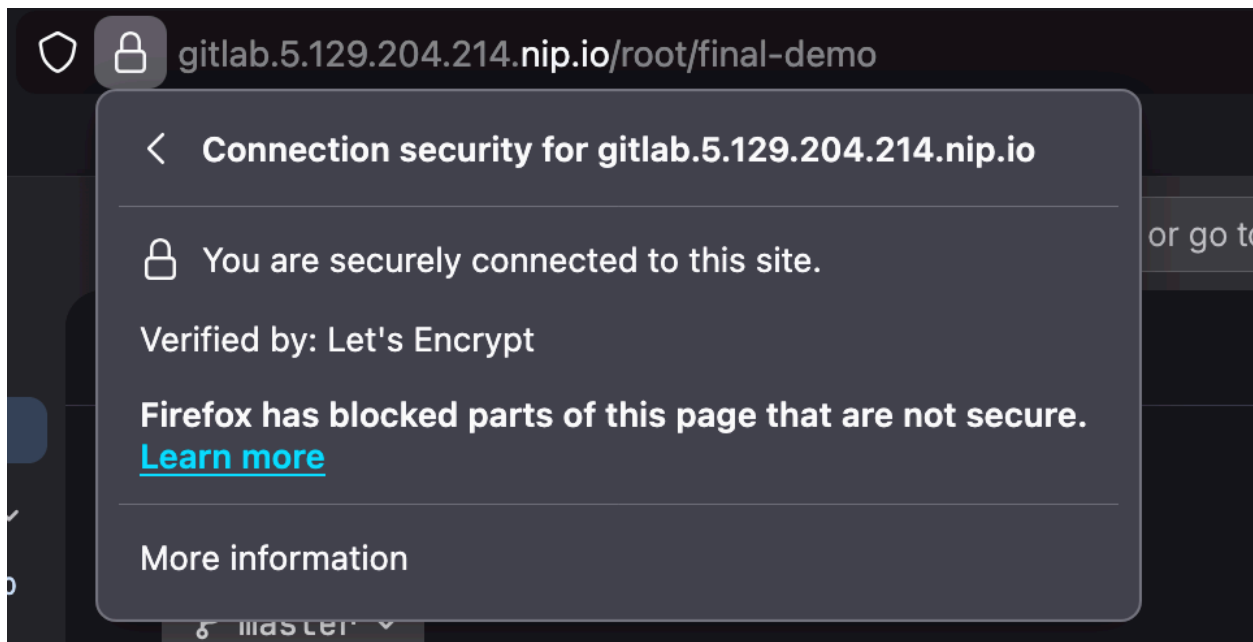


Figure 6.3— HTTPS working for GitLab via Caddy

7. LLM Integration into CI/CD Pipeline

The LLM is integrated as a dedicated `llm-review` stage in the GitLab CI/CD pipeline. It receives application source code or pipeline configuration and returns structured analysis covering code quality, security issues, and improvement suggestions.

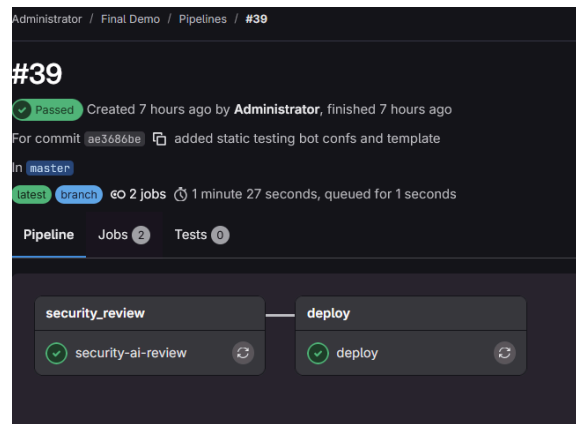


Figure 7.1 — CI/CD pipeline with a llm-security-review stage

7.1 Prompt Design

```

prompt = f"""
You are a senior application security
engineer performing an automated code
security audit.

Analyze the provided git diff for security
vulnerabilities.

Focus on:
- OWASP Top 10 vulnerabilities
- secrets or API keys
- SQL/command injection
- authentication flaws
- insecure crypto
- unsafe deserialization
- path traversal
- unsafe input handling
- privilege escalation risks

Return structured markdown:

### Risk Summary
LOW | MEDIUM | HIGH | CRITICAL

### Detected Issues
bullet points

### Secure Fix Recommendations
bullet points

### Developer Guidance
short explanation

File: {file}

Diff:
{diff}
"""

```

Figure 7.1 — CI/CD LLM prompt for security review

7.2 LLM API and Engine

Openrouter API was chosen because it provides free API key, variety of LLMs, auto-decision on the engine. Important to note that Openrouter has great documentation and pleasant UI to work with the engine.

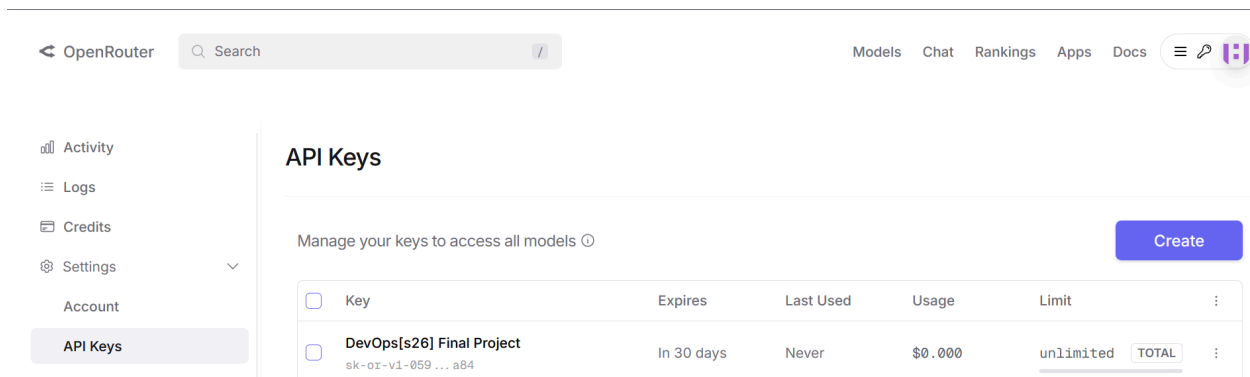


Figure 7.1 — OpenRouter API key setup

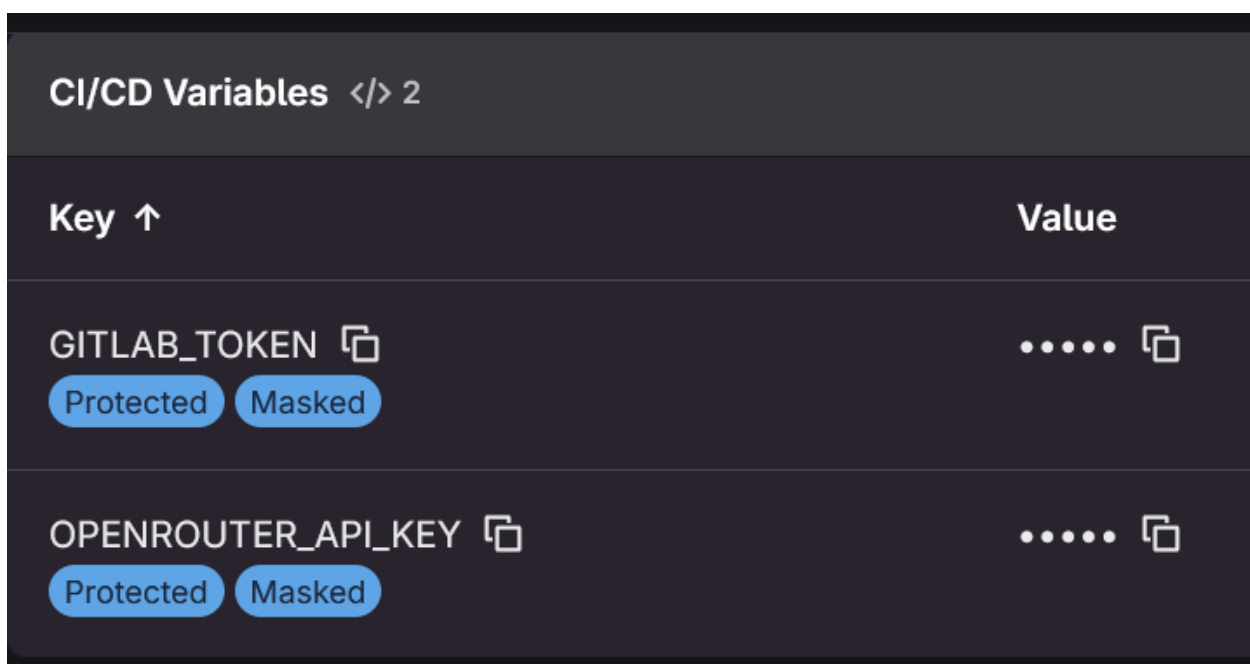


Figure 7.1 — Environment variables used for LLM features triggered on CI/CD pipeline

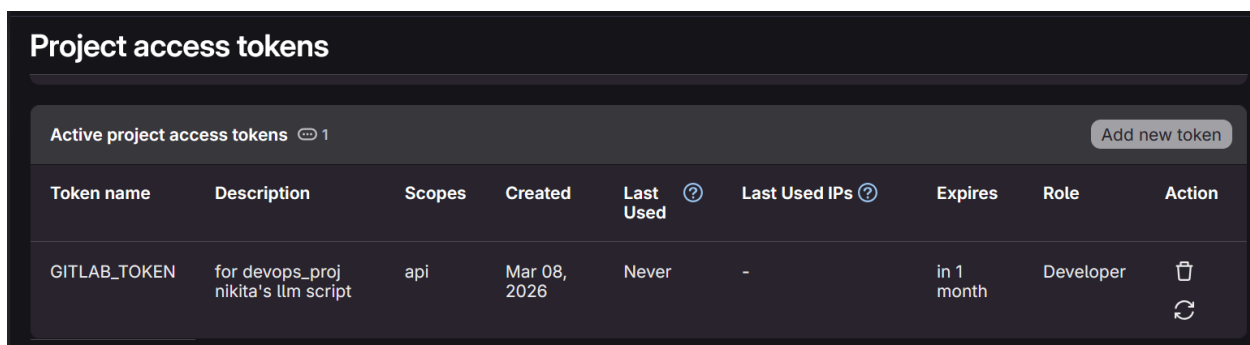


Figure 7.1 — Gitlab token foy python script used for LLM features triggered on CI/CD pipeline

7.3 Security Review Capabilities

The LLM is asked to scan git diffs for OWASP Top 10 vulnerabilities including SQL injection, command injection, path traversal, unsafe deserialization, and insecure cryptographic usage, along with detecting hardcoded secrets, API keys, and authentication flaws. It checks for unsafe input handling, privilege escalation risks, and insecure configurations that could appear in any committed code changes across infrastructure or application files. Each finding is classified by severity as LOW, MEDIUM, HIGH, or CRITICAL, with structured output providing detected issues, secure fix recommendations, and developer guidance.

Execution

Here are the pipeline stage configuration and LLM outputs. `.gitlab-ci.yml`

```
# .gitlab-ci.yml – llm-review stage
stages:
  ...
  - security_review
  ...

security-ai-review:
  stage: security_review
  image: python:3.11
  before_script:
    - pip install requests
    - apt-get update
    - apt-get install -y git
  script:
    - python ai_security_review.py
```

```
Administrator / Final Demo / Jobs / #156

Log timestamps in UTC.
336 09:14:09 Setting up libperl5.40:amd64 (5.40.1-6) ...
337 09:14:09 Setting up perl (5.40.1-6) ...
338 09:14:09 Setting up libgssapi-krb5-2:amd64 (1.21.3-5) ...
339 09:14:09 Setting up xauth (1:1.1.2-1.1) ...
340 09:14:09 Setting up libgnutls30t64:amd64 (3.8.9-3+deb13u2) ...
341 09:14:09 Setting up openssh-client (1:10.0p1-7) ...
342 09:14:09 Setting up libpsl5t64:amd64 (0.21.2-1.1+b1) ...
343 09:14:09 Setting up liberror-perl (0.17030-1) ...
344 09:14:09 Setting up librtmp1:amd64 (2.4+20151223.gitfa8646d.1-2+b5) ...
345 09:14:09 Setting up libngtcp2-crypto-gnutls8:amd64 (1.11.0-1) ...
346 09:14:09 Setting up libcurl3t64-gnutls:amd64 (8.14.1-2+deb13u2) ...
347 09:14:09 Setting up git (1:2.47.3-0+deb13u1) ...
348 09:14:09 Processing triggers for libc-bin (2.41-12+deb13u1) ...
349 09:14:09 $ python ai_security_review.py
350 09:15:08 Starting AI Security Review
351 09:15:08 Analyzing file: static-testing-bot/Dockerfile
352 09:15:08 Analyzing file: static-testing-bot/k8s/configmap.yaml
353 09:15:08 Analyzing file: static-testing-bot/k8s/deployment.yaml
354 09:15:08 Analyzing file: static-testing-bot/k8s/namespace.yaml
355 09:15:08 Analyzing file: static-testing-bot/k8s/networkpolicy.yaml
356 09:15:08 Analyzing file: static-testing-bot/k8s/service.yaml
357 09:15:08 Analyzing file: static-testing-bot/k8s/serviceaccount.yaml
358 09:15:08 Analyzing file: static-testing-bot/main.py
359 09:15:08 Analyzing file: static-testing-bot/requirements.txt
360 09:15:08 Posting review to GitLab commit
361 09:15:08 Security review complete
362 09:15:08 Cleaning up project directory and file based variables
363 09:15:08 Job succeeded
```

Figure 7.1 — llm-review stage in GitLab CI pipeline

GITLAB_TOKEN @project_1_bot_76f02d94a2f26e22ef01000a07e79789 10 seconds ago

Developer

Automated AI Security Review

Commit: 11b038796f8dc34861a8f5b38219950422ed405b

File: ai_security_review.py

Risk Summary

LOW | 0 | MEDIUM | 1 | HIGH | 0 | CRITICAL | 1

Detected Issues

- **CRITICAL:** Hardcoded API key (OPENROUTER_API_KEY) is present in the code. This is a critical security risk as it can be exposed to unauthorized parties.
- **MEDIUM:** Insecure error handling. The code prints the error response from the OpenRouter API, which may contain sensitive information. This can lead to information disclosure.

Secure Fix Recommendations

- **Remove hardcoded API key:** Store the API key securely using environment variables or a secrets management solution.
- **Improve error handling:** Instead of printing the error response, log it securely and return a generic error message to the user.

Developer Guidance

When handling API keys and error responses, it's essential to prioritize security. Hardcoding API keys can lead to unauthorized access, and insecure error handling can disclose sensitive information. Use secure storage solutions for API keys and implement robust error handling mechanisms to prevent information disclosure.

Generated automatically by CI security pipeline.

Figure 7.2 — LLM artifact output (security and code review report)

static-testing-bot/Dockerfile0 → 100644+44 -0

```
27 + COPY --from=builder /root/.local /root/.local
28 +
29 + # Copy application code
30 + COPY main.py .
31 +
32 + # Set environment variables
33 + ENV PATH=/root/.local/bin:$PATH
34 + ENV PYTHONUNBUFFERED=1
35 +
36 + # Health check
37 + HEALTHCHECK --interval=30s --timeout=10s --start-period=40s --retries=3 \
38 +   CMD python -c "import requests; requests.get('http://localhost:8000/health') || exit 1
39 +
40 + # Expose port
41 + EXPOSE 8000
42 +
43 + # Run application
44 + CMD ["python", "-m", "uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

static-testing-bot/k8s/configmap.yaml0 → 100644+9 -0

```
1 + apiVersion: v1
2 + kind: ConfigMap
3 + metadata:
4 +   name: static-testing-bot-config
5 +   namespace: static-testing-bot
6 + data:
7 +   LOG_LEVEL: "INFO"
8 +   SCAN_TIMEOUT: "3600"
9 +   MAX_CONCURRENT_SCANS: "5"
```

Final Demo / Commits / ae3686be

Files

Q Search (e.g. *.vue) (F)

static-testing-bot

Dockerfile+44 -0

k8s

configmap.yaml+9 -0

deployment.yaml+84 -0

namespace.yaml+6 -0

networkpolicy.yaml+38 -0

service.yaml+16 -0

serviceaccount.yaml+7 -0

main.py+128 -0

requirements.txt+5 -0

When building Docker images, it's essential to ensure that sensitive information is not stored in the image. Use virtual environments and avoid storing secrets or API keys in files that are copied into the image. Regularly update and patch base images to ensure that they remain secure. Additionally, review health check endpoints to ensure that they are properly secured and do not reveal sensitive information.

File: static-testing-bot/k8s/configmap.yaml

Risk Summary

LOW | 0 | 0 | 0

Detected Issues

- None

Secure Fix Recommendations

- None

Developer Guidance

The provided git diff for the `configmap.yaml` file does not introduce any security vulnerabilities. The file contains configuration settings for a Kubernetes ConfigMap, which is used to store non-confidential data. The values set for `LOG_LEVEL`, `SCAN_TIMEOUT`, and `MAX_CONCURRENT_SCANS` do not pose any security risks.

However, it's essential to note that ConfigMaps should not be used to store sensitive data, such as API keys or database credentials. If sensitive data needs to be stored, consider using a Secret resource instead.

As a best practice, ensure that the ConfigMap is properly labeled and annotated to follow the organization's security policies and compliance requirements. Additionally, regularly review and update the ConfigMap to ensure it remains secure and compliant.

Figure 7.3 — LLM security review on a Kubernetes configmap and dockerfile (up).

Final Project

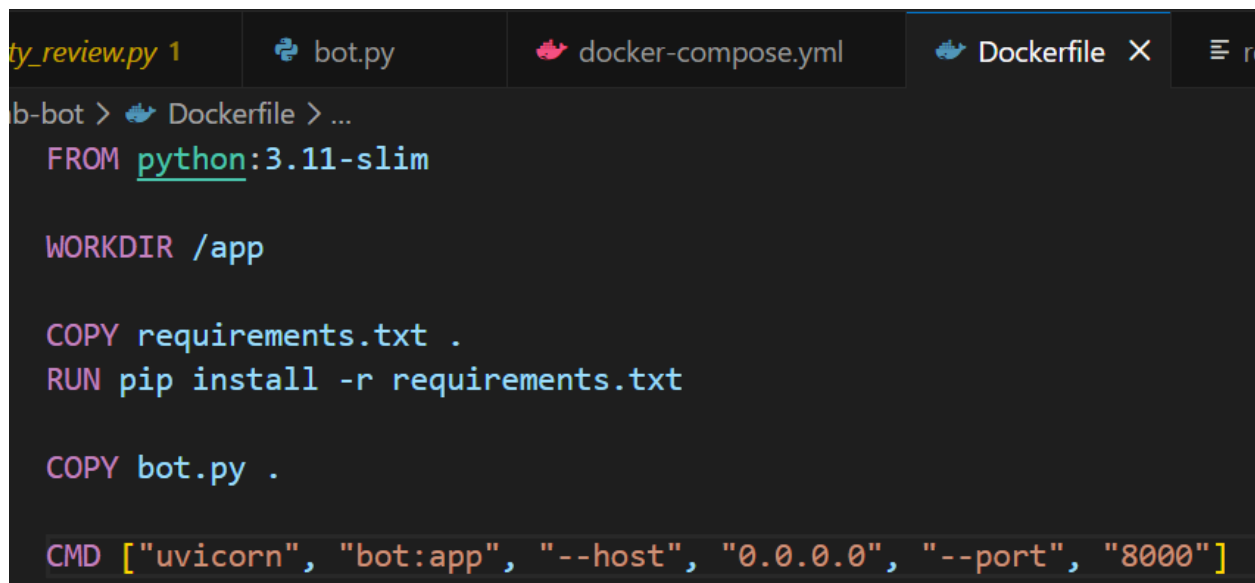
24

8. AI GitLab Bot — Web Service

The AI bot is a **Uvicorn-based Python web server** that integrates with GitLab via webhooks. It listens for slash commands posted in commits and responds with LLM-powered analysis.

8.1 Web Server Architecture

Uvicorn set on localhost inside k8s minikube in a few nodes. Port 8000 is used. Python 3.11-slim is used for light installation.

A screenshot of a code editor showing a Dockerfile configuration. The editor has tabs for 'ty_review.py 1', 'bot.py', 'docker-compose.yml', and 'Dockerfile'. The 'Dockerfile' tab is active, showing the following content: FROM python:3.11-slim, WORKDIR /app, COPY requirements.txt ., RUN pip install -r requirements.txt, COPY bot.py ., and CMD ["uvicorn", "bot:app", "--host", "0.0.0.0", "--port", "8000"].

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install -r requirements.txt

COPY bot.py .

CMD ["uvicorn", "bot:app", "--host", "0.0.0.0", "--port", "8000"]
```

Figure 8.1 — Dockerfile configuration for uvicorn server.

8.2 Bot Commands

The bot supports the following commands:

Command	Description
<code>/ai explain</code>	Reviews Terraform, Kubernetes manifests, and IaC scripts for insecure settings or open ports

Command	Description
<code>/ai "how's production?"</code>	Returns a summary of the current deployment health and recent pipeline status

8.3 GitLab Webhook Integration

The GitLab webhook is configured to send event payloads to the bot's `/ai-webhook` POST endpoint, with a secret token set via `X-Gitlab-Token` header that the bot compares against its `WEBHOOK_SECRET` environment variable. If the secret matches the bot proceeds to process the event, and if it doesn't match the bot returns an "unauthorized" status, though it notably returns HTTP 200 rather than a 401/403 status code.

Execution

Bellow is the bot code structure, webhook config.

```
"""
AI Security Bot – Code Skeleton / Template
Shows all main components: config, helpers, middleware, endpo
int.
"""

from fastapi import FastAPI, Request
import os, re, socket, requests

app = FastAPI()

# — Config —————
OPENROUTER_API_KEY = os.environ["OPENROUTER_API_KEY"]
GITLAB_TOKEN       = os.environ["GITLAB_TOKEN"]
PROJECT_ID         = os.environ["PROJECT_ID"]
GITLAB_API         = os.environ["GITLAB_API"]
WEBHOOK_SECRET     = os.environ.get("WEBHOOK_SECRET", "")
HOSTNAME           = socket.gethostname()
AI_MODEL           = "meta-llama/llama-3.1-70b-instruct"
```

```

MAX_PROMPT          = 5000

# — Helpers —————
def sanitize(text: str) -> str:
    """Strip secrets/keys/passwords from text before sending
    to LLM."""
    # ... regex redaction, then truncate to MAX_PROMPT ...
    return text[:MAX_PROMPT]

def ask_llm(prompt: str) -> str:
    """Send sanitized prompt to OpenRouter and return LLM res
    ponse."""
    # ... POST to openrouter.ai/api/v1/chat/completions ...
    # ... headers: Authorization, Content-Type, HTTP-Referer,
    X-Title ...
    # ... body: model, messages, temperature=0.2, max_tokens=
    1200 ...
    # ... fail safely on error, never leak secrets ...
    return "LLM response"

def get_commit_diff(commit: str) -> str:
    """Fetch diff for a commit from GitLab API."""
    # ... GET {GITLAB_API}/projects/{PROJECT_ID}/repository/c
    ommits/{commit}/diff ...
    # ... concatenate file paths + diffs, truncate to MAX_PRO
    MPT ...
    return "diff text"

def post_reply(commit: str, discussion_id: str, text: str) ->
None:
    """Post a note back to a GitLab commit discussion."""
    # ... POST {GITLAB_API}/projects/{PROJECT_ID}/.../discuss
    ions/{id}/notes ...
    pass

# — Middleware —————

```

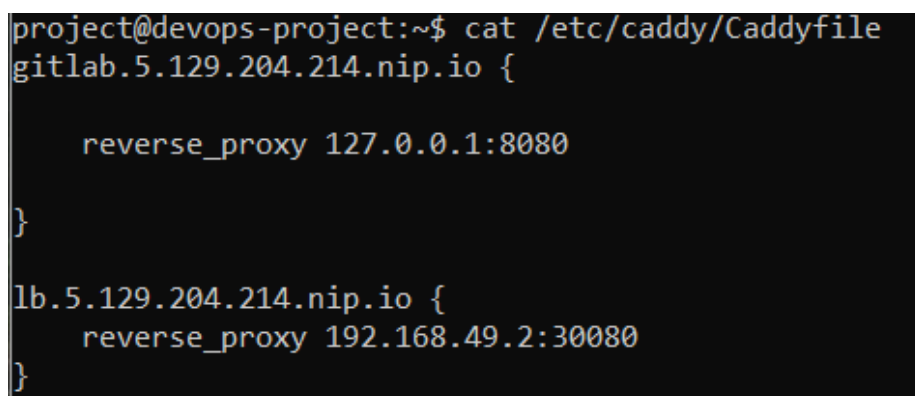
```

@app.middleware("http")
async def add_hostname_header(request: Request, call_next):
    """Attach X-Served-By header to every response."""
    response = await call_next(request)
    response.headers["X-Served-By"] = HOSTNAME
    return response

# — Webhook Endpoint —————
@app.post("/ai-webhook")
async def webhook(req: Request):
    """Main entry point: receives GitLab note events, runs A
    I, replies."""
    # 1. Verify WEBHOOK_SECRET via X-Gitlab-Token header
    # 2. Ignore non-"note" events
    # 3. Ignore notes not starting with "/ai"
    # 4. Extract commit ID from payload
    # 5. Build prompt (user command or default "explain")
    # 6. Fetch diff → ask LLM → post reply back to GitLab
    return {"status": "ok"}

```

As bot is served in Minikube on port 30080, it only accessible locally via minikube address. So we need to use proxy to forward external requests to minikube pods:



```

project@devops-project:~$ cat /etc/caddy/Caddyfile
gitlab.5.129.204.214.nip.io {
    reverse_proxy 127.0.0.1:8080
}

lb.5.129.204.214.nip.io {
    reverse_proxy 192.168.49.2:30080
}

```

Figure 8.1 — Caddy proxy configuration

Pods itself are running in minikube:

```
project@devops-project:~$ kubectl get pods -n lb-demo
NAME                                READY   STATUS    RESTARTS   AGE
ai-bot-57b97fdbd6-ql8qq             1/1     Running   0           26h
ai-bot-57b97fdbd6-v7mwv             1/1     Running   0           26h
ai-bot-57b97fdbd6-zj5zm             1/1     Running   0           26h
haproxy-7c69486459-qhmqf           1/1     Running   0           26h
```

Figure 8.2 — Running pods

Webhook configuration in project settings which allows bot answer directly in commit comments:

Administrator / Final Demo / Webhook settings / Webhook

Webhook

Name (optional)

Description (optional)

URL

The URL must be percent-encoded if it contains one or more special characters.

+ Add URL masking [?](#)

Secret token

Used to validate received payloads. Sent with the request in the **X-GitLab-Token** HTTP header.

Trigger

☐ Push events

☐ Tag push events
A new tag is pushed to the repository.

☒ Comments
A comment is made or edited on an issue or merge request.

[How to create a custom webhook template?](#)

SSL verification

☒ Enable SSL verification

Figure 8.3 — GitLab webhook configuration

Status	Trigger	Elapsed time	Request time	
200	Note Hook	0.04 sec	7 hours ago	View details
200	Note Hook	4.64 sec	7 hours ago	View details
200	Note Hook	0.09 sec	7 hours ago	View details
200	Note Hook	0.03 sec	7 hours ago	View details
200	Note Hook	5.63 sec	7 hours ago	View details

Figure 8.4 — Webhook logs of payloads

Administrator @root 12 hours ago

/ai explain

GITLAB_TOKEN @project_3_bot_a81f12dba6082dc9e7c425e0ea66dd91 12 hours ago

AI Bot served by ai-bot-57b97fdbd6-v7mwv

This commit sets up a new static testing bot application using Docker and Kubernetes. Here's a breakdown of what's changed:

Dockerfile:

- A new Dockerfile is created for the static testing bot application.
- The Dockerfile uses a multi-stage build process to create a lightweight image.
- The first stage installs build dependencies and copies the application code.
- The second stage installs runtime dependencies and copies the Python dependencies from the first stage.
- The final image is created with the application code and dependencies.

Figure 8.5 — /ai explain command triggered in GitLab MR

9. Deployment

The AI GitLab Bot is deployed behind **HAProxy** as a load balancer, distributing traffic across multiple Uvicorn worker instances for availability and performance.

All resources are deployed in a dedicated namespace "lb-demo", which isolates the load balancing demo from other workloads in the cluster.

Final pipeline configuration that deploys haproxy and ai-bot pods with security review:

```
stages:
  - security_review
  - deploy

security-ai-review:
  stage: security_review
  image: python:3.11-slim
  before_script:
    - pip install requests
    - apt-get update
    - apt-get install -y git
  script:
    - python ai_security_review.py

deploy:
  stage: deploy
  image: bitnami/kubectl:latest
  script:
    - kubectl apply -f k8s/namespace.yaml
    - kubectl apply -f k8s/bot.yaml
    - kubectl apply -f k8s/haproxy.yaml
    - kubectl rollout restart deployment/haproxy -n lb-demo
    - kubectl rollout status deployment/ai-bot -n lb-demo --t
imeout=60s
    - kubectl rollout status deployment/haproxy -n lb-demo --
timeout=60s

only:
  - master
```

9.1 HAProxy Load Balancer

ConfigMap haproxy-config stores the HAProxy configuration file. The configuration defines:

- HTTP mode with sensible timeouts (connect 5s, client 30s, server 30s);
- "option forwardfor" to preserve the original client IP;
- a single frontend bound to port 80 that routes all traffic to the ai-bot backend;
- round-robin load balancing algorithm, which distributes requests evenly across all available backend pods;
- active health checks (check) on each backend server — HAProxy automatically removes unhealthy pods from rotation and restores them when they recover

Deployment haproxy runs a single HAProxy pod using the official haproxy:alpine image. The configuration file is mounted from the ConfigMap via a volume, which allows updating the configuration without rebuilding the image.

Service haproxy exposes HAProxy externally via NodePort 30080, making the application accessible from outside the Minikube cluster at Minikube address <http://192.168.49.2:30080>. Internally it listens on port 80.

HAProxy distributes incoming requests across the three bot replicas using round-robin. Each response includes the "X-Served-By" header with the pod name, providing a clear visual proof that load balancing is working.

haproxy deployment configuration:

```
---
apiVersion: v1
kind: ConfigMap
metadata:
  name: haproxy-config
  namespace: lb-demo
data:
  haproxy.cfg: |
    global
      log stdout local0 info
```

```

defaults
  mode http
  timeout connect 5s
  timeout client 30s
  timeout server 30s
  option forwardfor
  option httplog
  log global

frontend http
  bind *:80
  default_backend ai-bot

backend ai-bot
  balance roundrobin
  server-template ai-bot 3 ai-bot-headless.lb-demo.svc.cl
uster.local:8000 check resolvers k8sdns resolve-prefer ipv4 i
nit-addr none
  resolvers k8sdns
    nameserver dns1 kube-dns.kube-system.svc.cluster.local:
53
    accepted_payload_size 8192
    hold valid 5s

---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: haproxy
  namespace: lb-demo
spec:
  replicas: 1
  selector:
    matchLabels:
      app: haproxy

```

```

template:
  metadata:
    labels:
      app: haproxy
  spec:
    containers:
      - name: haproxy
        image: haproxy:alpine
        ports:
          - containerPort: 80
        volumeMounts:
          - name: config
            mountPath: /usr/local/etc/haproxy/haproxy.cfg
            subPath: haproxy.cfg
    volumes:
      - name: config
        configMap:
          name: haproxy-config
---
apiVersion: v1
kind: Service
metadata:
  name: haproxy
  namespace: lb-demo
spec:
  type: NodePort
  selector:
    app: haproxy
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30080

```

9.2 Bot Deployment

Deployment ai-bot runs 3 replicas of the AI GitLab bot, built from a custom Docker image ai-gitlab-bot:latest. Key configuration points:

- imagePullPolicy: Never instructs Kubernetes to use the locally built image inside Minikube rather than pulling from a registry;
- all sensitive configuration (API keys, tokens) is injected via envFrom referencing a Kubernetes Secret ai-bot-secret, keeping credentials out of the image and the repository;
- the application listens on port 8000 (uvicorn/FastAPI);
- each pod exposes its own hostname via the X-Served-By response header, making it possible to observe which pod handled each request.

Service ai-bot is a ClusterIP service (internal only) that load balances traffic across all 3 bot replicas on port 8000. HAProxy uses this service as its backend target.

ai-bot deployment configuration:

```
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ai-bot
  namespace: lb-demo
spec:
  replicas: 3
  selector:
    matchLabels:
      app: ai-bot
  template:
    metadata:
      labels:
        app: ai-bot
    spec:
      containers:
        - name: ai-bot
```

```

        image: ai-gitlab-bot:latest
        imagePullPolicy: Never
        ports:
          - containerPort: 8000
        envFrom:
          - secretRef:
              name: ai-bot-secret
    ---
apiVersion: v1
kind: Service
metadata:
  name: ai-bot-headless
  namespace: lb-demo
spec:
  clusterIP: None
  selector:
    app: ai-bot
  ports:
    - port: 8000
      targetPort: 8000

```

9.3 Proof of Concept

Checking running pods and services:

```

project@devops-project:~$ kubectl get pods -n lb-demo
NAME                                READY   STATUS    RESTARTS   AGE
ai-bot-57b97fdbd6-2h6h4             1/1     Running   0           17h
ai-bot-57b97fdbd6-v7mwv             1/1     Running   0           44h
ai-bot-57b97fdbd6-zj5zm             1/1     Running   0           44h
haproxy-644b88d66-2rx74             1/1     Running   0           16m
project@devops-project:~$ kubectl get svc -n lb-demo
NAME                TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
ai-bot-headless     ClusterIP   None         <none>        8000/TCP         26m
haproxy             NodePort    10.108.120.127 <none>        80:30080/TCP     5d19h

```

Figure 10.1 — Running pods and services

Making sure that load balancing is working, by receiving different "X-Server-By" header value for different requests:

```
project@devops-project:~$ curl -I -X POST https://lb.5.129.204.214.nip.io/ai-webhook
HTTP/2 200
alt-svc: h3=":443"; ma=2592000
content-type: application/json
date: Sat, 14 Mar 2026 15:23:38 GMT
server: uvicorn
via: 1.1 Caddy
x-served-by: ai-bot-57b97fdbd6-2h6h4
content-length: 25

project@devops-project:~$ curl -I -X POST https://lb.5.129.204.214.nip.io/ai-webhook
HTTP/2 200
alt-svc: h3=":443"; ma=2592000
content-type: application/json
date: Sat, 14 Mar 2026 15:23:39 GMT
server: uvicorn
via: 1.1 Caddy
x-served-by: ai-bot-57b97fdbd6-v7mwv
content-length: 25

project@devops-project:~$ curl -I -X POST https://lb.5.129.204.214.nip.io/ai-webhook
HTTP/2 200
alt-svc: h3=":443"; ma=2592000
content-type: application/json
date: Sat, 14 Mar 2026 15:23:39 GMT
server: uvicorn
via: 1.1 Caddy
x-served-by: ai-bot-57b97fdbd6-zj5zm
content-length: 25
```

Figure 10.2 — Load balancing in action via curl

Also we can check pod name directly in bot's answer in Gitlab comments:

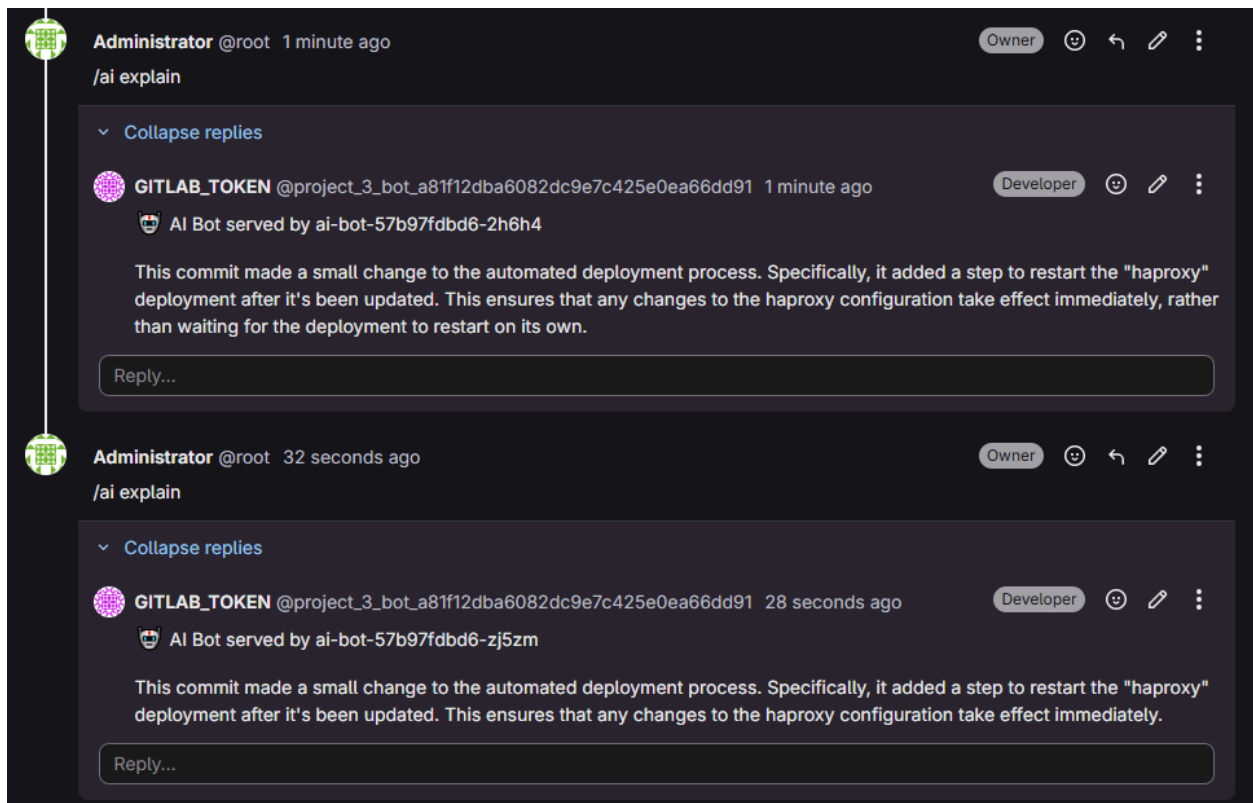


Figure 10.3 — Visible pod name in Gitlab comments

Additionally we can check fault tolerance of the deployment by deleting one of the pods.

After that kubernetes will automatically run new pod, and other pods are still working, bot is still working without interruptions:

```

project@devops-project:~$ kubectl delete pod ai-bot-57b97fdbd6-ql8qq -n lb-demo
pod "ai-bot-57b97fdbd6-ql8qq" deleted from lb-demo namespace
project@devops-project:~$ kubectl get pods -n lb-demo
NAME                                READY   STATUS    RESTARTS   AGE
ai-bot-57b97fdbd6-2h6h4             1/1     Running   0           6s
ai-bot-57b97fdbd6-v7mwv             1/1     Running   0           27h
ai-bot-57b97fdbd6-zj5zm             1/1     Running   0           27h
haproxy-7c69486459-qhmqf           1/1     Running   0           27h
project@devops-project:~$ curl -I https://lb.5.129.204.214.nip.io/ai-webhook
HTTP/2 405
allow: POST
alt-svc: h3=":443"; ma=2592000
content-type: application/json
date: Fri, 13 Mar 2026 22:04:54 GMT
server: uvicorn
via: 1.1 Caddy
x-served-by: ai-bot-57b97fdbd6-zj5zm
content-length: 31

```

Figure 10.4 — Fault tolerance check

For troubleshooting purposes we can also check logs directly from pods, and debug if needed:

```

project@devops-project:~$ kubectl logs -n lb-demo -l app=ai-bot
INFO:    Waiting for application startup.
INFO:    Application startup complete.
INFO:    Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO:    10.244.0.153:52668 - "POST /ai-webhook HTTP/1.1" 200 OK
INFO:    10.244.0.164:34322 - "POST /ai-webhook HTTP/1.1" 200 OK
INFO:    10.244.0.164:34322 - "POST /ai-webhook HTTP/1.1" 200 OK
INFO:    10.244.0.164:34166 - "POST /ai-webhook HTTP/1.1" 200 OK
Reply posted: 201
INFO:    10.244.0.164:38854 - "POST /ai-webhook HTTP/1.1" 200 OK
INFO:    10.244.0.164:49390 - "POST /ai-webhook HTTP/1.1" 200 OK
INFO:    10.244.0.153:41892 - "POST /ai-webhook HTTP/1.1" 200 OK
INFO:    10.244.0.153:41892 - "POST /ai-webhook HTTP/1.1" 200 OK
INFO:    10.244.0.153:36784 - "GET / HTTP/1.1" 404 Not Found
INFO:    10.244.0.153:47648 - "POST /ai-webhook HTTP/1.1" 200 OK
INFO:    10.244.0.153:47648 - "POST /ai-webhook HTTP/1.1" 200 OK
INFO:    10.244.0.153:47648 - "POST /ai-webhook HTTP/1.1" 200 OK
INFO:    10.244.0.164:42020 - "POST /ai-webhook HTTP/1.1" 200 OK
INFO:    10.244.0.164:42020 - "POST /ai-webhook HTTP/1.1" 200 OK
INFO:    10.244.0.164:42286 - "POST /ai-webhook HTTP/1.1" 200 OK
INFO:    10.244.0.164:53824 - "POST /ai-webhook HTTP/1.1" 200 OK
INFO:    10.244.0.153:58968 - "HEAD /ai-webhook HTTP/1.1" 405 Method Not Allowed
INFO:    10.244.0.153:58968 - "HEAD /ai-webhook HTTP/1.1" 405 Method Not Allowed
INFO:    10.244.0.153:59624 - "GET /robots.txt HTTP/1.1" 404 Not Found
INFO:    10.244.0.153:50412 - "POST /ai-webhook HTTP/1.1" 200 OK
INFO:    10.244.0.164:51160 - "POST /ai-webhook HTTP/1.1" 200 OK
INFO:    10.244.0.164:51160 - "POST /ai-webhook HTTP/1.1" 200 OK
INFO:    10.244.0.164:51160 - "POST /ai-webhook HTTP/1.1" 200 OK
INFO:    10.244.0.164:36416 - "POST /ai-webhook HTTP/1.1" 200 OK
Reply posted: 201
INFO:    10.244.0.164:60298 - "POST /ai-webhook HTTP/1.1" 200 OK

```

Figure 10.5 — ai-bot pods logs

```

project@devops-project:~$ kubectl logs -n lb-demo -l app=haproxy
<134>Mar 14 15:09:31 haproxy[9]: 10.244.0.1:54789 [14/Mar/2026:15:09:31.059] http ai-bot/ai-bot2 0/0/0/2
/2 200 183 - - ---- 1/1/0/0/0 0/0 "POST /ai-webhook HTTP/1.1"
<134>Mar 14 15:09:31 haproxy[9]: 10.244.0.1:54789 [14/Mar/2026:15:09:31.515] http ai-bot/ai-bot3 0/0/0/1
/1 200 183 - - ---- 1/1/0/0/0 0/0 "POST /ai-webhook HTTP/1.1"
<134>Mar 14 15:09:31 haproxy[9]: 10.244.0.1:54789 [14/Mar/2026:15:09:31.794] http ai-bot/ai-bot1 0/0/0/2
/2 200 183 - - ---- 1/1/0/0/0 0/0 "POST /ai-webhook HTTP/1.1"
<134>Mar 14 15:23:38 haproxy[9]: 10.244.0.1:50094 [14/Mar/2026:15:23:38.837] http ai-bot/ai-bot2 0/0/2/6
/8 200 183 - - ---- 1/1/0/0/0 0/0 "POST /ai-webhook HTTP/1.1"
<134>Mar 14 15:23:39 haproxy[9]: 10.244.0.1:50094 [14/Mar/2026:15:23:39.231] http ai-bot/ai-bot3 0/0/0/2
/2 200 183 - - ---- 1/1/0/0/0 0/0 "POST /ai-webhook HTTP/1.1"
<134>Mar 14 15:23:39 haproxy[9]: 10.244.0.1:50094 [14/Mar/2026:15:23:39.764] http ai-bot/ai-bot1 0/0/1/2
/3 200 183 - - ---- 1/1/0/0/0 0/0 "POST /ai-webhook HTTP/1.1"
<134>Mar 14 15:24:32 haproxy[9]: 10.244.0.1:16992 [14/Mar/2026:15:24:29.574] http ai-bot/ai-bot2 0/0/3/2
476/2480 200 173 - - ---- 1/1/0/0/0 0/0 "POST /ai-webhook HTTP/1.1"
<134>Mar 14 15:24:32 haproxy[9]: 10.244.0.1:16992 [14/Mar/2026:15:24:32.431] http ai-bot/ai-bot3 0/0/1/2
/3 200 178 - - ---- 1/1/0/0/0 0/0 "POST /ai-webhook HTTP/1.1"
<134>Mar 14 15:25:13 haproxy[9]: 10.244.0.1:18476 [14/Mar/2026:15:25:10.154] http ai-bot/ai-bot1 0/0/2/3
480/3483 200 173 - - ---- 1/1/0/0/0 0/0 "POST /ai-webhook HTTP/1.1"
<134>Mar 14 15:25:13 haproxy[9]: 10.244.0.1:18476 [14/Mar/2026:15:25:13.959] http ai-bot/ai-bot2 0/0/0/2
/2 200 178 - - ---- 1/1/0/0/0 0/0 "POST /ai-webhook HTTP/1.1"
project@devops-project:~$

```

Figure 10.6 — haproxy logs

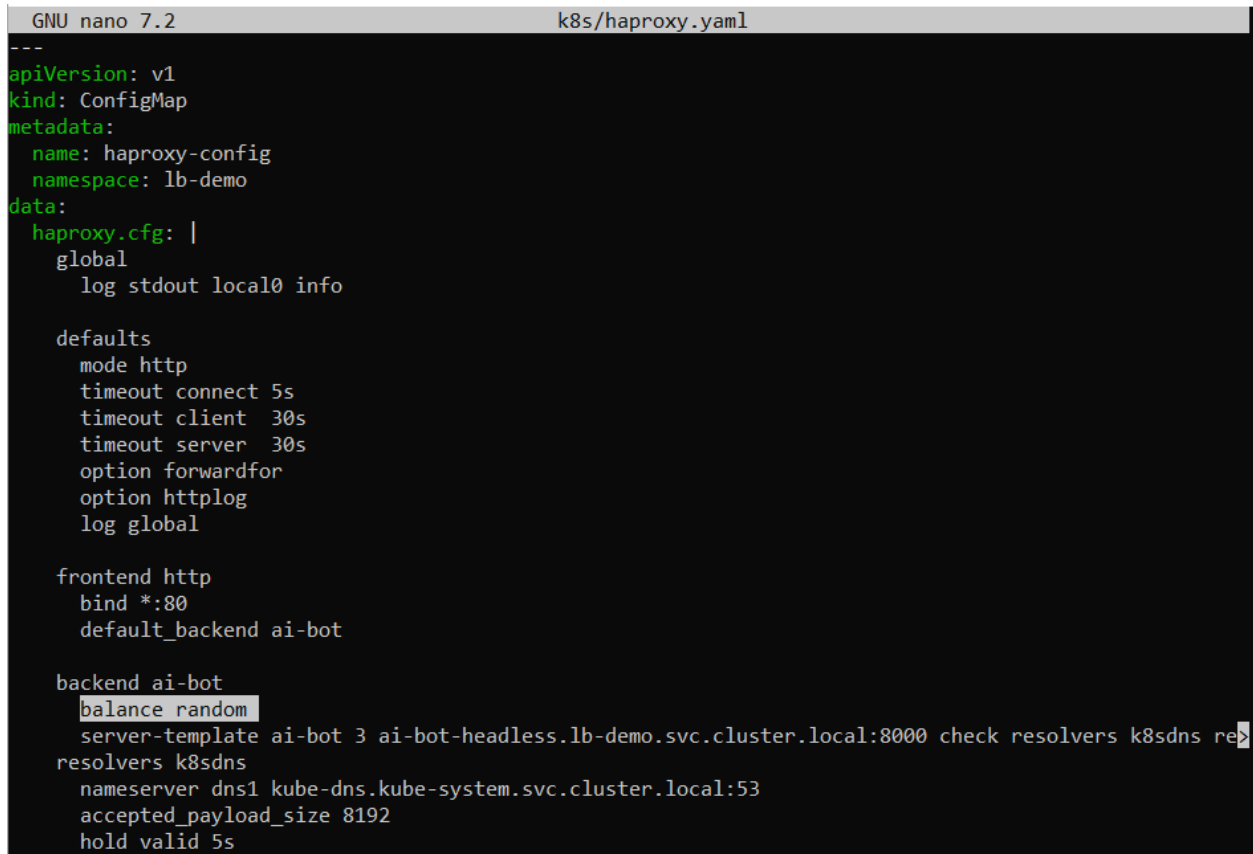
9.4 Changing balancing algorithm

HAProxy supports multiple load balancing algorithms, each suited for different use cases. Round Robin distributes requests evenly across all backend servers in sequence, which works well when all servers have equal capacity and request processing time is the same. Least Connections (leastconn) routes each new request to the server with the fewest active connections, making it a better choice for workloads with variable response times such as LLM API calls. Random selects a backend server randomly, which statistically produces similar distribution to round robin but without maintaining any state. Source hashes the client IP to always route the same client to the same server, useful when session persistence is required.

For our demo environment, Round Robin is the most appropriate choice. All three bot replicas are identical, run on the same node, and handle stateless webhook requests. Since the goal is to visually demonstrate load balancing where each request lands on a different pod in a predictable, observable sequence round robin provides the clearest proof of concept. The X-Served-By response header confirms that requests are distributed evenly across ai-bot pods, which is exactly what we need to validate the setup.

But we still able to configure our deployment and change balancing algorithm. For example, we can change "round-robin" to "random" algorithm in haproxy configuration and deploy it, which is also suitable for out small environment and can be easily demonstrated.

Changed value of "balance" parameter in haproxy configuration in the haproxy's ConfigMap



```
GNU nano 7.2 k8s/haproxy.yaml
---
apiVersion: v1
kind: ConfigMap
metadata:
  name: haproxy-config
  namespace: lb-demo
data:
  haproxy.cfg: |
    global
      log stdout local0 info

    defaults
      mode http
      timeout connect 5s
      timeout client 30s
      timeout server 30s
      option forwardfor
      option httplog
      log global

    frontend http
      bind *:80
      default_backend ai-bot

    backend ai-bot
      balance random
      server-template ai-bot 3 ai-bot-headless.lb-demo.svc.cluster.local:8000 check resolvers k8sdns re
      resolvers k8sdns
        nameserver dns1 kube-dns.kube-system.svc.cluster.local:53
        accepted_payload_size 8192
        hold valid 5s
```

Figure 10.7 — Updated balancing algorithm in Configmap

Deployed it to the cluster:

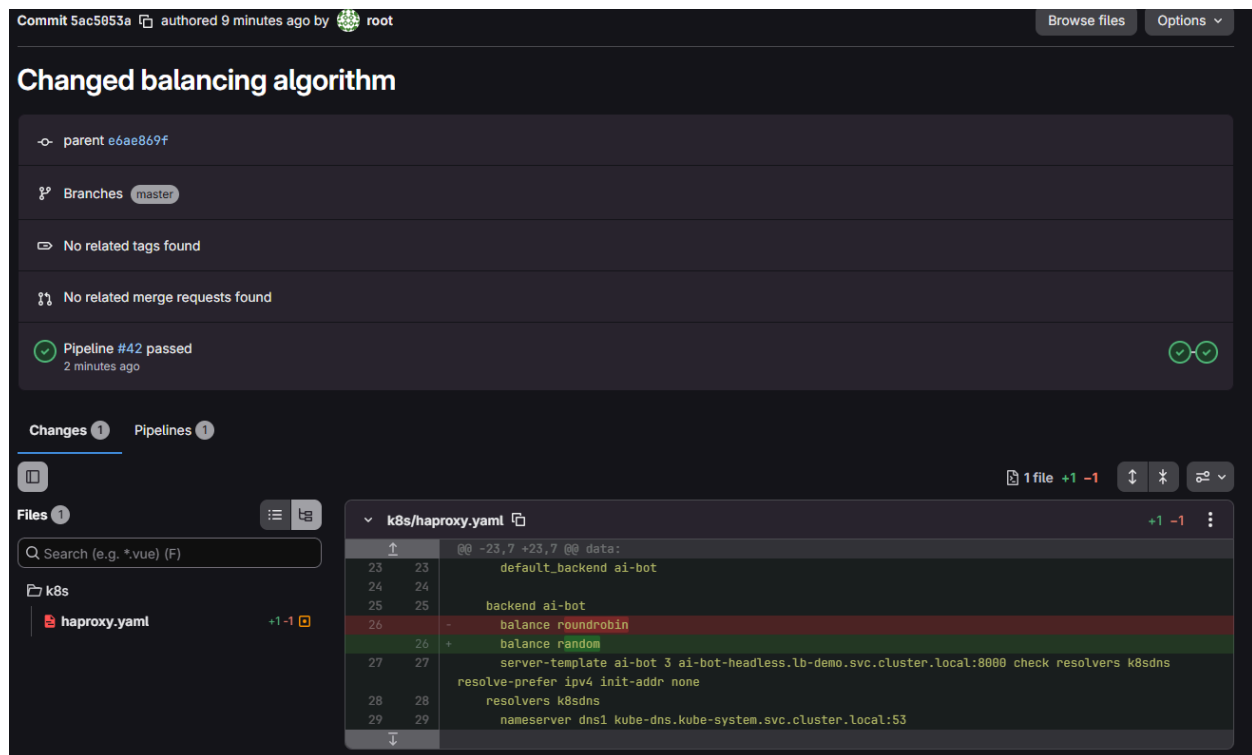


Figure 10.8 — Committing changes

Checking requests balancing via curl:

```

project@devops-project:~/final-demo$ curl -I -X POST https://lb.5.129.204.214.nip.io/ai-webhook
HTTP/2 200
alt-svc: h3=":443"; ma=2592000
content-type: application/json
date: Sat, 14 Mar 2026 15:44:37 GMT
server: uvicorn
via: 1.1 Caddy
x-served-by: ai-bot-57b97fdbd6-v7mwv
content-length: 25

project@devops-project:~/final-demo$ curl -I -X POST https://lb.5.129.204.214.nip.io/ai-webhook
HTTP/2 200
alt-svc: h3=":443"; ma=2592000
content-type: application/json
date: Sat, 14 Mar 2026 15:44:39 GMT
server: uvicorn
via: 1.1 Caddy
x-served-by: ai-bot-57b97fdbd6-2h6h4
content-length: 25

project@devops-project:~/final-demo$ curl -I -X POST https://lb.5.129.204.214.nip.io/ai-webhook
HTTP/2 200
alt-svc: h3=":443"; ma=2592000
content-type: application/json
date: Sat, 14 Mar 2026 15:44:40 GMT
server: uvicorn
via: 1.1 Caddy
x-served-by: ai-bot-57b97fdbd6-v7mwv
content-length: 25

project@devops-project:~/final-demo$ kubectl get pods -n lb-demo
NAME                                READY   STATUS    RESTARTS   AGE
ai-bot-57b97fdbd6-2h6h4             1/1     Running   0           17h
ai-bot-57b97fdbd6-v7mwv             1/1     Running   0           44h
ai-bot-57b97fdbd6-zj5zm             1/1     Running   0           44h
haproxy-6c5b75c548-fj4zv           1/1     Running   0           99s
project@devops-project:~/final-demo$ nano k8s/haproxy.yaml

```

Figure 10.9 — Checking balancing via curl

10. Future Development

Describe growth points and potential improvements identified during the project.

- ☐ Extend bot commands to support `/ai review` for full MR diff analysis
- ☐ Integrate SAST results directly into LLM prompts for context-aware security advice
- ☐ Add support for self-hosted LLM (e.g. llama3 via Ollama) to avoid sending code to external APIs
- ☐ Implement Kubernetes HPA (Horizontal Pod Autoscaler) for the bot under load

- ☐ Add persistent logging of all LLM interactions for audit purposes
 - ☐ Explore GitLab AI-native features and compare with custom implementation
-

11. References

- [GitLab CE Documentation](#)
- [Kubernetes Documentation](#)
- [Caddy Documentation](#)
- [HAProxy Documentation](#)
- [Fail2ban Documentation](#)
- [OpenAI API Documentation](#)
- [FastAPI Documentatio](#)